Escuela Superior Politécnica de Chimborazo
Facultad de Informática y Electrónica
Carrera de Software

CODIFICAMED: CODIFICACIÓN AUTOMÁTICA CIE-10 DE HISTORIAS CLÍNICAS CON PROCESAMIENTO DE LENGUAJE NATURAL APLICANDO LA METODOLOGÍA CRISP-ML(Q)

Nombre:
Bryan Jefferson Guayan Pandashina

Docente:
Ing. Isaac David Torres Paredes

Asignatura:
Base de Conocimiento

Riobamba, 3 de noviembre del 2026

Tabla de Contenido

1 Descripción del Proyecto....2
2 Objetivo General....2
3 Desarrollo....3
3.1 Fase 1: Comprensión del Negocio y de los Datos (Business and Data Understanding)….3
3.2 Fase 2: Ingeniería de Datos (Data Engineering)....4
3.3 Fase 3: Ingeniería del Modelo (Machine Learning Model Engineering)....5
3.4 Fase 4: Aseguramiento de Calidad del Modelo (Evaluation)....7
3.5 Fase 5: Despliegue (Deployment)....7
3.6 Fase 6: Monitoreo y Mantenimiento (Monitoring and Maintenance)....9
4 Arquitectura de la Solución....10
5 Resultados y Comparación: Modelo Local vs. API Externa....11
6 Conclusiones y Limitaciones....13
7 Enlaces....14
8 Referencias....14

1 Descripción del Proyecto

Este proyecto implementa CodificaMed, un prototipo que recibe una nota clínica escrita en texto libre y sugiere los códigos de la Clasificación Internacional de Enfermedades, décima revisión (CIE-10), mostrando la frase de la nota que justifica cada código. En la Red Pública Integral de Salud (MSP, IESS) y en las clínicas privadas del Ecuador la codificación es manual: un codificador lee notas con abreviaturas ("HTA", "DM2"), errores de tipeo y negaciones ("niega fiebre", "se descarta neumonía") y asigna los códigos uno por uno. Un código mal asignado genera glosas, retrasa la facturación y produce estadísticas de morbilidad poco confiables.

Se adoptó la metodología CRISP-ML(Q) (Cross-Industry Standard Process for Machine Learning with Quality assurance) en sus seis fases, cada una cerrada con una verificación de calidad. El sistema combina un pipeline de PLN basado en reglas (expansión de abreviaturas, corrección ortográfica y detección de negación NegEx adaptada al español), recuperación semántica de códigos candidatos sobre el catálogo oficial CIE-10 (RAG) y un modelo de lenguaje que se cambia por configuración: local con Ollama o externo por API. Con la API externa (gpt-oss-120b en Groq) alcanzó un F1 de 0,923 sobre 40 casos clínicos ficticios, sin confirmar ningún diagnóstico negado, y todo se ejecutó con herramientas gratuitas.

2 Objetivo General

Desarrollar un prototipo que sugiera códigos CIE-10 a partir de notas clínicas en texto libre, con la evidencia de cada código y la distinción entre diagnósticos confirmados, negados y en sospecha, aplicando las seis fases de la metodología CRISP-ML(Q) y comparando un modelo de lenguaje local con una API externa mediante precisión, recall y F1.

3 Desarrollo

3.1 Fase 1: Comprensión del Negocio y de los Datos *(Business and Data Understanding)*

Actividades realizadas:

- **Definición de objetivos del negocio**:
  - Reducir el tiempo de codificación: el sistema sugiere y el codificador humano solo valida.
  - Disminuir las glosas por códigos mal asignados, en especial los diagnósticos negados que se codifican como confirmados.
- **Evaluación de la situación**:
  - **Recursos disponibles**: Python 3.14, MacBook Air M4 con 16 GB de RAM, Ollama, librerías open-source (sentence-transformers, scikit-learn, RapidFuzz, Streamlit) y el plan gratuito de la API de Groq.
  - **Restricciones**: prohibido usar historias clínicas reales, costo $0 y ejecución con modelo local y con API externa cambiando solo la configuración.
  - **Riesgos**: abreviaturas ambiguas ("EDA" puede ser enfermedad diarreica aguda o endoscopía digestiva alta), negaciones y la diferencia entre la CIE-10 de la OMS (4 caracteres, la que usa el MSP) y la CIE-10-ES del corpus CodiEsp (hasta 7 caracteres).
- **Determinación de objetivos de aprendizaje automático**:
  - **Tarea**: clasificación multietiqueta de cada nota en códigos CIE-10, con evidencia y estado (confirmado, negado o sospecha).
  - **Métricas**: precisión, recall y F1 por código exacto y por categoría de 3 caracteres, diagnósticos negados confirmados por error, latencia y tokens.
- **Comprensión de los datos**:
  - **Catálogo**: CIE-10 de la OMS en español (lista tabular MINSAL/DEIS 2008) con 14 247 categorías y subcategorías.
  - **Casos clínicos**: corpus público CodiEsp (CLEF eHealth 2020) como fuente opcional y casos ficticios redactados para el proyecto.
- **Elaboración del plan del proyecto**:

- **Metodología**: CRISP-ML(Q) en 6 fases.
- **Enfoque técnico**: reglas de PLN + recuperación semántica (RAG) + modelo de lenguaje.
- **Herramientas**: Python, Ollama (qwen2.5:7b), Groq (gpt-oss-120b) y Streamlit.

Implementación en el código:

```
# .env  (el proveedor se cambia sin tocar el código)
LLM_PROVIDER=ollama             # ollama | openai | anthropic | rag
OLLAMA_MODEL=qwen2.5:7b
OPENAI_MODEL=openai/gpt-oss-120b
OPENAI_BASE_URL=https://api.groq.com/openai/v1
EMBEDDING_BACKEND=hibrido
NEGACION_POLITICA=hibrida
```

3.2 Fase 2: Ingeniería de Datos *(Data Engineering)*

Actividades realizadas:

- **Construcción del catálogo CIE-10**:
  - **Extracción**: 14 247 categorías (E11) y subcategorías (E11.9) del archivo oficial. Se excluyeron las causas externas (V01-Y98) y quedaron 10 553 códigos indexables.
  - **Texto de búsqueda enriquecido**: cada subcategoría hereda la descripción de su categoría padre; "E11.9 Sin mención de complicación" pasa a "Diabetes mellitus no insulinodependiente: Sin mención de complicación".
- **Base de conocimiento**:
  - **Sinónimos clínicos**: 90 entradas de uso local ("hipertensión arterial" para I10, "DM2" para E11, "lumbalgia" para M54.5).
  - **Siglas**: diccionario de 86 abreviaturas con reglas de contexto ("EDA:" seguida de dos puntos se expande como endoscopía digestiva alta).
- **Conjunto de evaluación**:
  - **Casos ficticios**: 40 notas con abreviaturas, errores de tipeo ("gonartosis", "se decarta"), negaciones y diagnósticos descartados.
  - **Etiquetas**: 58 códigos esperados (con alternativas válidas, p. ej. J44.0 o J44.1), códigos opcionales y 40 códigos negados que no deben confirmarse.

- **Verificación de la calidad de los datos**:
    - **Validación automática**: todos los códigos de referencia deben existir en el catálogo. La prueba detectó y corrigió dos códigos que solo existen en CIE-10-ES (N18.4 y B98.0).

Implementación en el código:

```
# data/casos_ficticios.jsonl (un caso por línea)
{"id": "F01", "texto": "Pcte masculino de 58 años con antecedentes de HTA y
  DM2 en tto con metformina y losartán. ... Niega fiebre, niega dolor
  torácico. ... Dx: HTA controlada. DM2 sin complicaciones.",
 "codigos": ["I10", "E11.9"], "opcionales": [], "negados": ["R50.9",
"R07.4"]}

# scripts/build_catalog.py
def texto(row):
    if len(row.codigo) == 3 or not row.descripcion_categoria:
        return row.descriptor
    return f"{row.descripcion_categoria}: {row.descriptor}"
```

3.3 Fase 3: Ingeniería del Modelo *(Machine Learning Model Engineering)*

Actividades realizadas:

- **Preprocesamiento del texto clínico**:
    - **Limpieza**: conserva la posición de cada carácter para resaltar luego la evidencia sobre la nota original.
    - **Segmentación**: en oraciones (respetando "38.2" o "120.000") y en cláusulas, que son la unidad de búsqueda.
    - **Normalización**: expansión de siglas, corrección ortográfica por distancia de edición contra el vocabulario del catálogo ("gonartosis" → "gonartrosis") y eliminación del relleno ("paciente de 58 años con…").
- **Detección de negación (NegEx-es)**:
    - **Disparadores**: 43 frases previas y posteriores ("niega", "se descarta", "sin", "BHCG negativa") y de sospecha ("probable", "a descartar").
    - **Alcance**: se corta con terminadores ("pero", ":", ";") y se ignoran falsas negaciones como "no especificado".
- **Representación (búsqueda híbrida)**:

- **Embeddings**: multilingual-e5-small (384 dimensiones) sobre el catálogo enriquecido, combinados en partes iguales con TF-IDF de n-gramas de caracteres.
- **Justificación**: e5 solo ubicaba I11, I12 e I13 por delante de I10 para "hipertensión arterial"; la búsqueda híbrida devuelve I10 en primer lugar (Figura 1).

- **Modelado (RAG + LLM)**:
  - **Recuperación**: cada cláusula recupera sus 8 códigos más similares y se envían hasta 60 candidatos al LLM junto con la nota marcada por NegEx ("niega [NEG fiebre]").
  - **Generación y validación**: el LLM devuelve un JSON con código, evidencia literal y estado. La salida se valida contra el catálogo y, si el LLM confirma algo que NegEx ve negado, NegEx lo corrige (política híbrida).
  - **Proveedores**: local qwen2.5:7b en Ollama; API externa openai/gpt-oss-120b en Groq (plan gratuito, compatible con OpenAI). También se soporta Claude y una línea base sin LLM.

Implementación en el código:

```
# codificamed/recuperador.py
def similitudes(self, consultas):
    if self._partes:                        # búsqueda híbrida e5 + TF-IDF
        densa, lexica = (p.similitudes(consultas) for p in self._partes)
        return 0.5 * densa + 0.5 * lexica

# codificamed/negacion.py
("se descarta", NEGADO, "pre"), ("niega", NEGADO, "pre"), ("sin", NEGADO,
"pre"),
("negativa", NEGADO, "post"), ("probable", SOSPECHA, "pre"), ...
```

Figura 1

Construcción del índice híbrido: "hipertensión arterial" recupera I10 en primer lugar

```
Índice hibrido listo: 10553 códigos en 1.0 s
W1103 18:43:38.949000 27653 .venv/lib/python3.14/site-packages/torch/distributed/elastic/multiproces
Loading weights: 100%|██████████████████████████████████████████████████████████| 199/199 [00:00<00
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable hig
  'hipertensión arterial'                -> [('I10', 0.83), ('I15.0', 0.74), ('I15', 0.73)]
  'diabetes mellitus tipo 2'             -> [('E13', 0.87), ('O24', 0.83), ('E11', 0.82)]
  'neumonía adquirida en la comunidad'   -> [('J18.9', 0.85), ('J18', 0.8), ('J18.1', 0.78)]
```

3.4 Fase 4: Aseguramiento de Calidad del Modelo *(Evaluation)*

Actividades realizadas:

- **Protocolo de evaluación**:
  - Mismos 40 casos, mismo índice y mismos candidatos para todos los proveedores, con temperatura 0.
  - Un acierto cuenta si el código confirmado coincide con el esperado o con una alternativa válida; los códigos opcionales no suman ni restan.
- **Métricas calculadas**:
  - Precisión, recall y F1 (micro y macro) por código exacto y por categoría de 3 caracteres.
  - Negados confirmados por error, evidencia ubicada en la nota, latencia por nota y tokens consumidos.
- **Configuraciones comparadas**:
  - Línea base solo RAG, modelo local con y sin corrección de NegEx, y API externa. Los resultados se presentan en la sección 5.
- **Pruebas automáticas**:
  - 17 pruebas unitarias (negación, siglas, segmentación, métricas y pipeline completo con un servidor Ollama simulado), todas aprobadas.

Implementación en el código:

```
python scripts/evaluar.py --provider rag                   # línea base
python scripts/evaluar.py --provider ollama                # modelo local
python scripts/evaluar.py --provider openai                # API externa
(Groq)
python scripts/evaluar.py --provider ollama --negacion llm  # sin corrección
NegEx
python scripts/comparar.py                                 # tabla y gráfico
```

3.5 Fase 5: Despliegue *(Deployment)*

Actividades realizadas:

- **Implementación de la interfaz (Streamlit)**:
  - **Pestañas**: codificar nota, comparar local vs. API, evaluación y acerca de.

- **Evidencia visual**: la nota se resalta en verde (confirmado), rojo (negado) y ámbar (sospecha), con el código junto a cada frase (Figuras 2 y 3).
- **Validación humana**: el codificador acepta o rechaza cada código y descarga la codificación validada en JSON.

- **Cambio de proveedor**: desde la barra lateral o con LLM_PROVIDER en .env, sin tocar el código.
- **Instalación reproducible**: requirements.txt, .env.example sin claves y README paso a paso; el índice de embeddings se guarda en caché y la aplicación arranca en segundos.

Implementación en el código:

```
python scripts/construir_indice.py   # precalcula el índice híbrido
streamlit run app.py                 # abre http://localhost:8501
```

Figura 2

Caso F01 codificado con el modelo local (Ollama, qwen2.5:7b)

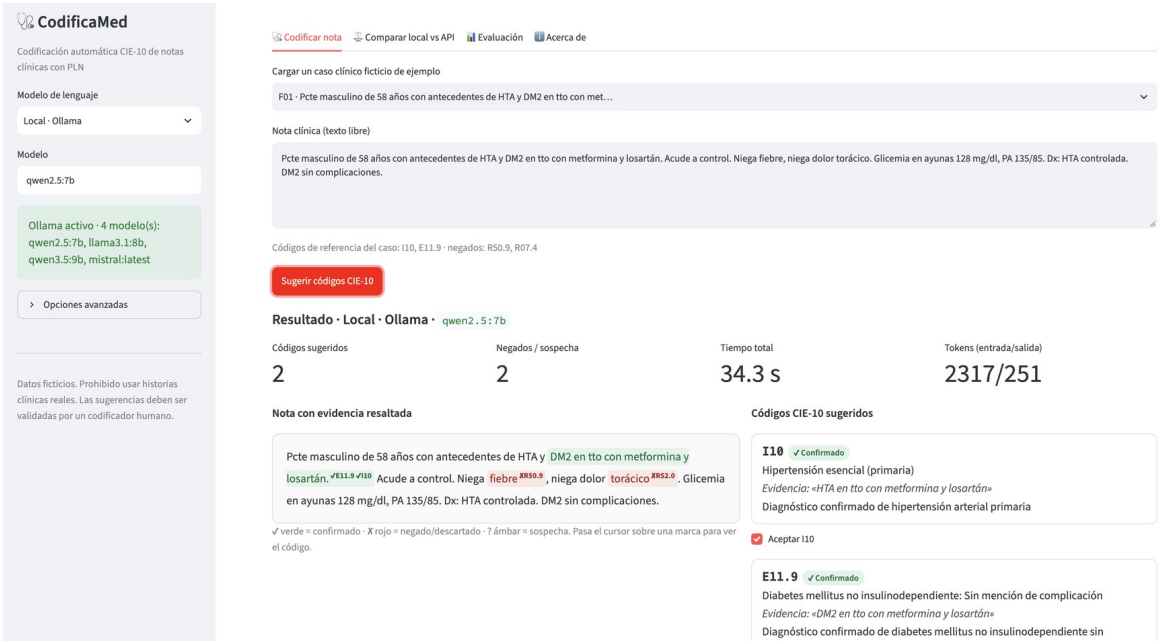
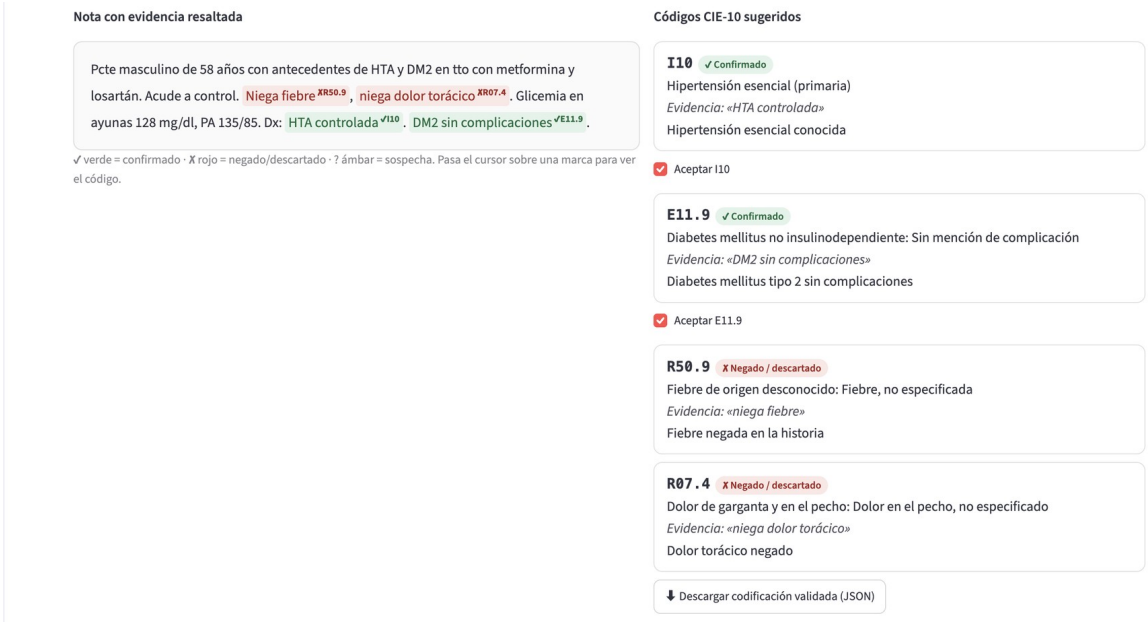


Figura 3

Caso F01 con la API externa: I10 y E11.9 confirmados, R50.9 y R07.4 negados



3.6 Fase 6: Monitoreo y Mantenimiento *(Monitoring and Maintenance)*

Actividades realizadas:

- **Planificación del seguimiento**:
  - **Monitoreo**: cada corrida guarda predicciones y métricas en results/, y comparar.py consolida la tabla y el gráfico para detectar regresiones.
  - **Alertas**: la interfaz avisa de códigos fuera de los candidatos, evidencia no ubicada y desacuerdos entre el LLM y NegEx.
  - **Mantenimiento**: el sistema se amplía sin reentrenar, agregando sinónimos, siglas o disparadores de negación en archivos de datos.
- **Revisión del proyecto**:
  - **Éxitos**: los seis requisitos de la tarea cumplidos, F1 de 0,923 con la API y ningún negado confirmado en ninguna configuración.
  - **Lecciones aprendidas**: un ejemplo dentro del prompt puede ser copiado por el modelo como respuesta, y los proveedores retiran modelos (Groq retiró llama-3.3-70b-versatile durante el desarrollo). Gracias a la configuración por .env, el cambio a gpt-oss-120b fue una sola línea.

Implementación en el código:

```
# Cambio de modelo de la API externa sin modificar el código
sed -i '' 's#^OPENAI_MODEL=.*#OPENAI_MODEL=openai/gpt-oss-120b#' .env
```

4 Arquitectura de la Solución

La solución es un pipeline de cinco etapas (Figura 4). Solo la etapa 4 depende del proveedor de LLM, así que el modelo local y la API externa reciben los mismos candidatos y la misma nota marcada; esto hace justa la comparación. El índice y el proveedor son las dos piezas reemplazables: el índice crece con más sinónimos y el proveedor se cambia en .env.

Figura 4

Arquitectura de CodificaMed: cinco etapas y dos fuentes reemplazables

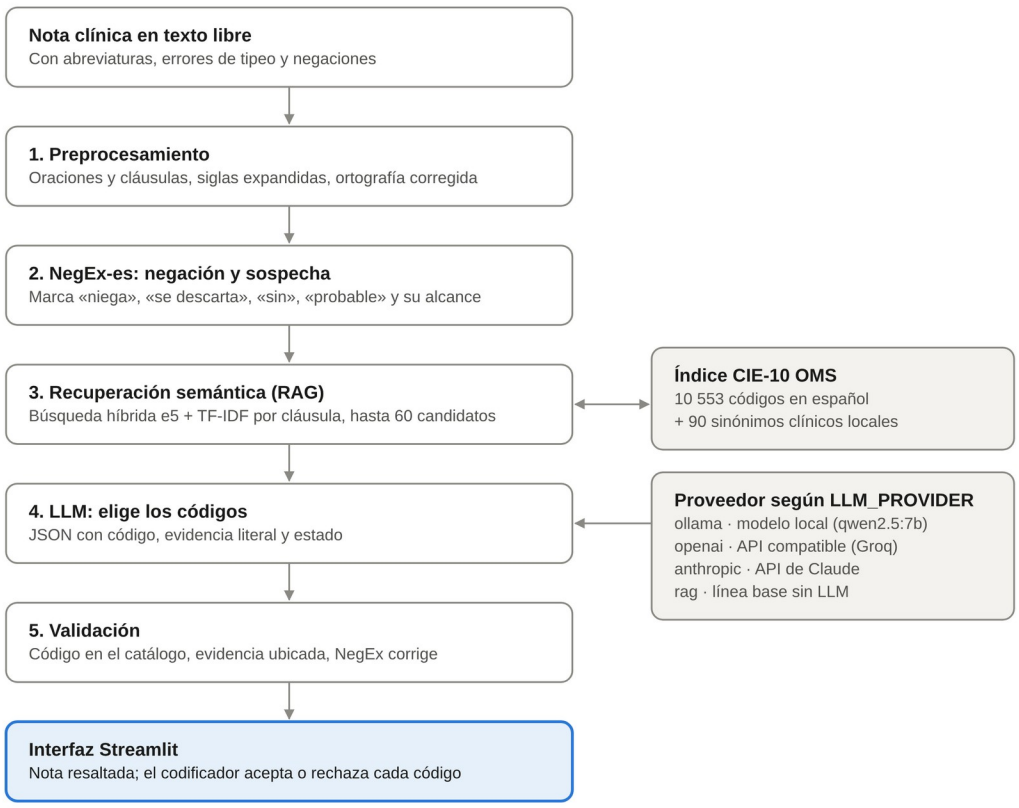


Tabla 1

Componentes de la solución

Componente	Módulo	Tecnología
Configuración	config.py, .env	python-dotenv
Catálogo y base de conocimiento	catalog.py, abreviaturas.py, sinonimos_cie10.csv	pandas
Preprocesamiento	preprocesamiento.py	Expresiones regulares, RapidFuzz
Negación	negacion.py	NegEx adaptado al español
Recuperación (RAG)	recuperador.py	sentence-transformers (e5), scikit-learn (TF-IDF)
Modelo de lenguaje	llm/	Ollama, SDK de OpenAI (Groq), SDK de Anthropic
Orquestación y validación	pipeline.py, prompts.py	Python 3.14
Evaluación	evaluacion.py, evaluar.py, comparar.py	matplotlib
Interfaz	app.py	Streamlit

5 Resultados y Comparación: Modelo Local vs. API Externa

La API externa (gpt-oss-120b en Groq) alcanzó un F1 de 0,923 por código exacto, frente a 0,506 del modelo local qwen2.5:7b y 0,440 de la línea base sin LLM (Tabla 2 y Figura 5). Ninguna configuración confirmó alguno de los 40 diagnósticos negados, y las cuatro corridas procesaron los 40 casos sin fallos.

Tabla 2

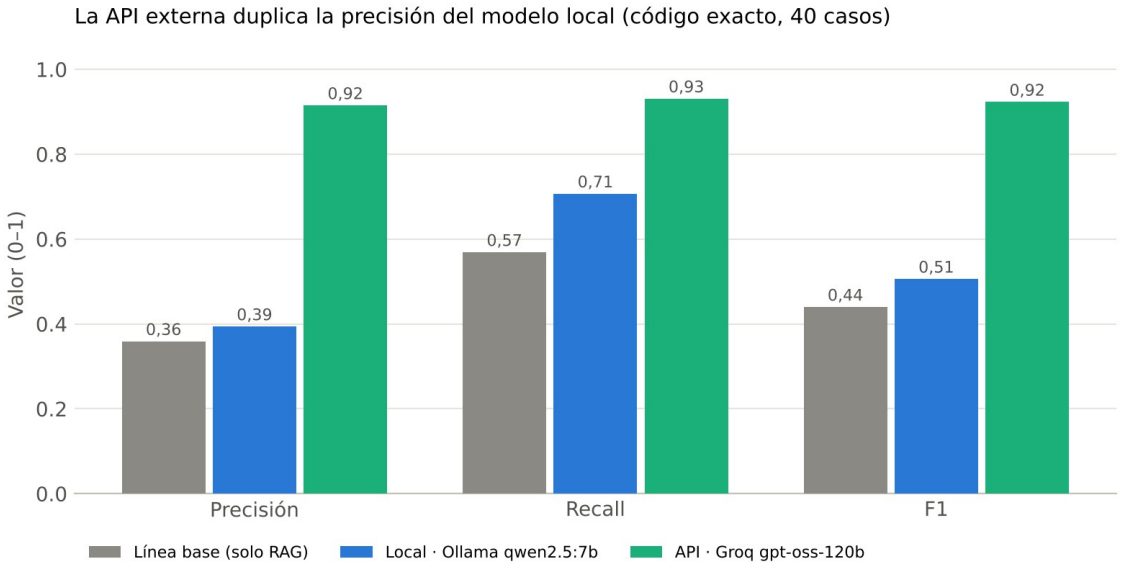
Resultados sobre los 40 casos clínicos ficticios

Configuración	P	R	F1	F1 categoría	Negados confirmados	Latencia por nota	Tokens entrada / salida
API externa (Groq gpt-oss-120b), NegEx híbrido	0,915	0,931	**0,923**	0,974	0 / 40	16,0 s	69 831 / 29 195
Local (Ollama qwen2.5:7b), NegEx híbrido	0,394	0,707	**0,506**	0,625	0 / 40	21,4 s	75 946 / 10 042
Local (Ollama qwen2.5:7b), solo LLM	0,394	0,707	**0,506**	0,625	0 / 40	21,4 s	75 946 / 10 042
Línea base, solo RAG (e5 + TF-IDF)	0,359	0,569	**0,440**	0,581	0 / 40	0,17 s	0 / 0

Nota. P, R y F1 por código exacto. Corridas del 3 de noviembre de 2026 en una MacBook Air M4 de 16 GB, con búsqueda híbrida y temperatura 0. La latencia de la API incluye las esperas por el límite de peticiones del plan gratuito de Groq; una nota aislada tardó 8,2 s.

Figura 5

Precisión, recall y F1 por configuración (código exacto)



Local frente a API. La API supera al modelo local en 42 puntos de F1, y la diferencia está sobre todo en la precisión (0,915 frente a 0,394). El modelo qwen2.5:7b agrega síntomas que el diagnóstico ya explica, aunque el prompt lo prohíbe: R35 (poliuria) en F02 y F17, R51 (cefalea) en F05 y F08, R50.8 (fiebre) en F07 y F38. También elige subcódigos vecinos equivocados: N42.9 en lugar de N40 (F26), F41.8 en lugar de F41.0 (F40) e I15.9 en lugar de I10 (F30). A cambio, el modelo local no envía datos fuera del equipo ni tiene costo por token.

Errores de la API. Son pocos y casi todos de especificidad: T30.2 en lugar de T22.2 en una quemadura de segundo grado (F37), K26.7 en lugar de K26.9 (F10), la categoría sin subcategoría en F28 (S81) y F13 (N18), y un código de más en F12, Z57.2 (exposición ocupacional al polvo).

Aporte de NegEx. Las dos políticas de negación dieron exactamente los mismos resultados: el modelo local nunca confirmó un diagnóstico negado, así que la corrección híbrida no tuvo que intervenir. En ambas políticas el LLM recibe la nota ya marcada por NegEx; medir cuánto aportan esas marcas requeriría una corrida adicional sin ellas.

Línea base. Sin un LLM que filtre, la recuperación sola encuentra el 74 % de las categorías esperadas, pero confirma síntomas y códigos parecidos, con una precisión de 0,36.

6 Conclusiones y Limitaciones

Sobre la construcción del proyecto en base a una metodología: la aplicación de las seis fases de CRISP-ML(Q) dio un marco que conectó cada decisión técnica con un objetivo medible. La verificación de calidad al cierre de cada fase detectó problemas a tiempo: códigos de referencia inexistentes en la OMS (fase 2), la debilidad de los embeddings solos para "hipertensión arterial" (fase 3) y un modelo retirado por el proveedor (fase 6). Gracias a que el proveedor se definió por configuración desde la fase 1, la comparación entre el modelo local y la API se hizo en condiciones idénticas y el cambio de modelo costó una línea.

Sobre el contenido en sí del proyecto: combinar reglas y modelos funcionó mejor que cualquiera de los dos por separado. NegEx y las siglas resuelven lo predecible y explicable, la búsqueda híbrida reduce 10 553 códigos a unos 60 candidatos y el LLM decide entre ellos con el contexto de la nota. La API externa alcanzó un F1 de 0,923 y fue además más rápida que el modelo local; el modelo local, con F1 de 0,506, queda como la opción para datos sensibles porque no envía información fuera del equipo. En todas las configuraciones el codificador recibe la evidencia de cada código y ningún diagnóstico negado fue confirmado.

Limitaciones:

- **Datos**: 40 casos ficticios redactados por el autor, más cortos y limpios que una nota real; las métricas no se extrapolan a la práctica clínica sin más validación.
- **Versiones de la CIE-10**: CodiEsp usa CIE-10-ES; al reducir sus códigos a la versión OMS se pierde detalle, por lo que allí la métrica confiable es la de categoría. El catálogo de 2008 no incluye actualizaciones como U07.1 (COVID-19).
- **Negación por reglas**: NegEx no capta negaciones implícitas o de largo alcance.
- **Dependencia del LLM**: el modelo puede proponer un código válido pero incorrecto; la decisión final debe tomarla un codificador.
- **Privacidad**: con la API externa la nota sale del equipo; con datos reales solo sería aceptable el modelo local o un acuerdo de tratamiento de datos.

7 Enlaces

Código fuente:

https://github.com/guayanjefferson10-cmd/codificamed

Video demostrativo:

https://liveespochedu-my.sharepoint.com/:v:/g/personal/bryan_guayan_espoch_edu_ec/IQC5d006F4K3SIjh6vLxrna2Ac5LdkIWLzzZnijp4lWOA_Q?e=GqF1xy

8 Referencias

Chapman, W. W., Bridewell, W., Hanbury, P., Cooper, G. F. y Buchanan, B. G. (2001). A simple algorithm for identifying negated findings and diseases in discharge summaries. *Journal of Biomedical Informatics, 34*(5), 301–310.

Miranda-Escalada, A., Gonzalez-Agirre, A., Armengol-Estapé, J. y Krallinger, M. (2020). Overview of automatic clinical coding: Annotations, guidelines, and solutions for non-English clinical cases at CodiEsp track of CLEF eHealth 2020. *CLEF 2020 Working Notes*.

Organización Mundial de la Salud. (2008). *Clasificación estadística internacional de enfermedades y problemas relacionados con la salud, 10.ª revisión (CIE-10)*. Lista tabular en español, MINSAL/DEIS.

Studer, S., Bui, T. B., Drescher, C., Hanuschkin, A., Winkler, L., Peters, S. y Müller, K.-R. (2021). Towards CRISP-ML(Q): A machine learning process model with quality assurance methodology. *Machine Learning and Knowledge Extraction, 3*(2), 392–413.

Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R. y Wei, F. (2024). Multilingual E5 text embeddings: A technical report. *arXiv:2402.05672*.